

Lesson 7.5 — Server Components + Authentication

Now we're connecting everything we've learned to **Next.js Server Components**.

The key question is:

If a Server Component runs on the server, how can it figure out which user is logged in?

The answer is surprisingly simple:

```
Server Component
  ↓
cookies()
  ↓
sessionId
  ↓
session store / DB
  ↓
current user
```



Let's build it from first principles.

7.5.1 — Why Server Components are naturally good for auth

Remember what we learned earlier:

A Server Component executes on the server.

```
export default async function DashboardPage() {  
  // This runs on the server  
}
```

That means it can safely access things that shouldn't be exposed to the browser, such as:

Database
Environment variables
Server-side session
Secrets
Private APIs



So we can do:

```
export default async function DashboardPage() {  
  const user = await getCurrentUser();  
  
  return <h1>Welcome, {user.name}</h1>;  
}
```

There is **no need to send the session ID to the Client Component just to determine who the user is.**

7.5.2 — Let's trace the request

Suppose the user has already logged in.

Their browser contains:

Cookie:
sessionId=ABC123



Now they request:

/dashboard



The flow is:

Browser

|



```
GET /dashboard
Cookie: sessionId=ABC123
▼
Next.js Server
  |
  ▼
Dashboard Server Component
  |
  ▼
getCurrentUser()
  |
  ▼
cookies()
  |
  ▼
sessionId = ABC123
  |
  ▼
Database / session store
  |
  ▼
ABC123 → userId 42
  |
  ▼
User 42
```

Then the Server Component can render:

```
<h1>Welcome, Gopi</h1>
```

7.5.3 — `cookies()` is the bridge

This is an important Next.js API:

```
import { cookies } from "next/headers";
```

Inside server-side code:

```
const cookieStore = await cookies();

const sessionId = cookieStore.get("sessionId")?.value;
```

So:

```
Cookie
  ↓
cookies()
  ↓
sessionId
```



The Server Component can therefore identify the current session.

7.5.4 — Our `getCurrentUser()` helper

We previously created:

```
export async function getCurrentUser() {
  const cookieStore = await cookies();

  const sessionId = cookieStore.get("sessionId")?.value;

  if (!sessionId) {
    return null;
  }

  const session = await db.session.findUnique({
    where: {
      id: sessionId,
    },
    include: {
      user: true,
    },
  });

  if (!session) {
    return null;
  }

  if (session.expiresAt < new Date()) {
    return null;
  }

  return session.user;
}
```

Now our Server Component becomes very clean:

```
export default async function DashboardPage() {
  const user = await getCurrentUser();

  if (!user) {
    redirect("/login");
  }

  return (
    <main>
      <h1>Welcome, {user.name}</h1>
    </main>
  );
}
```

7.5.5 — What actually crosses the browser/server boundary?

This is **very important** given everything you've learned about RSC. 🙄

Suppose the server finds:

```
user = {
  id: 42,
  name: "Gopi",
  email: "gopi@example.com"
}
```

The Server Component renders:

```
<h1>Welcome, Gopi</h1>
```

The browser does **not** receive:

```
Database connection
session lookup
password hash
session ID
```



Those remain server-side.

The browser receives the resulting UI/RSC information necessary to render the page.

So:

SERVER



```
cookies()  
session  
DB  
passwordHash  
private data  
auth logic
```

```
↓  
RSC Payload  
↓  
Browser
```

This is one of the major advantages of Server Components for authenticated applications.

7.5.6 — Do we need to make `getCurrentUser()` a Client Component?

✗ No.

In fact, you generally don't want to.

This would be wrong:

```
"use client";  
  
const user = await getCurrentUser();
```

because your authentication helper uses:

```
cookies()
```

and potentially:


```
</> TypeScript
```



Those are server-side concerns.

Instead:

```
// Server Component

const user = await getCurrentUser();
```

Perfectly natural.

7.5.7 — What if the page has interactive UI?

Now things get interesting.

Suppose:

```
/dashboard
```



contains:

```
Welcome Gopi 🙌

[Edit Profile]      ← interactive
[Delete Account]    ← interactive
```



You might have:

```
export default async function DashboardPage() {
  const user = await getCurrentUser();

  if (!user) {
    redirect("/login");
  }

  return (
    <>
      <h1>Welcome {user.name}</h1>

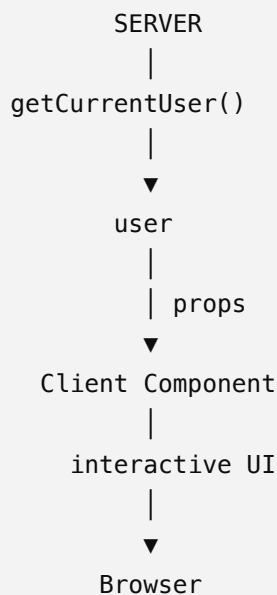
      <ProfileActions user={user} />
    </>
  );
}
```

```
);  
}
```

And:

```
"use client";  
  
export function ProfileActions({ user }) {  
  // useState  
  // onClick  
  // interactive UI  
}
```

Now we have:



This is perfectly valid.

7.5.8 — But should we send the whole user object?

Not necessarily.

This is a **security/data-minimization** consideration.

Suppose your database user is:

```
{  
  id: 42,  
  name: "Gopi",  
}
```

```
email: "gopi@example.com",
passwordHash: "...",
internalNotes: "...",
salary: "...",
resetToken: "..."
}
```

You obviously don't want to pass all of that:

```
<ClientComponent user={user} />
```

Instead, select only what the client needs:

```
const user = {
  id: dbUser.id,
  name: dbUser.name,
  email: dbUser.email,
};
```

Then:

```
<ProfileActions user={user} />
```

This is a good general rule:

Keep sensitive data server-side and send the client only what it actually needs.

7.5.9 — Very important: UI hiding is NOT authorization

Suppose the user isn't an admin.

You might do:

```
{user.role === "ADMIN" && (
  <DeleteUserButton />
)}
```

That's useful for the UI.

But it is **not security**.

A malicious user could still manually send:

```
DELETE /api/users/42
```



So the actual server-side operation must check authorization:

```
const user = await getCurrentUser();

if (!user) {
  throw new Error("Unauthorized");
}

if (user.role !== "ADMIN") {
  throw new Error("Forbidden");
}

await deleteUser(targetUserId);
```

The mental model:

```
Client
  ↓
UX restriction

Server
  ↓
REAL security boundary
```



7.5.10 — Server Component vs Server Action

This distinction is extremely important.

Server Component

Usually:

Read/render data

Example:

```
export default async function Dashboard() {  
  const user = await getCurrentUser();  
  
  return <DashboardUI user={user} />;  
}
```

Server Action

Usually:

Perform a mutation

Example:

```
"use server";  
  
export async function deleteAccount() {  
  const user = await getCurrentUser();  
  
  if (!user) {  
    throw new Error("Unauthorized");  
  }  
  
  await db.user.delete({  
    where: {  
      id: user.id,  
    },  
  });  
}
```

Both run on the server.

Both can authenticate the user.

But they serve different purposes.

7.5.11 — What if someone calls a Server Action directly?

This is another very important security concept.

Imagine your UI has:

```
<form action={deleteAccount}>  
  <button>Delete Account</button>
```

</form>

You might think:

"Only my UI can call this."

✗ No.

A Server Action is still an entry point into server-side behavior.

Therefore the action itself must authenticate and authorize:

```
"use server";

export async function deleteAccount() {
  const user = await getCurrentUser();

  if (!user) {
    throw new Error("Unauthorized");
  }

  // authorization/business rules

  await db.user.delete({
    where: {
      id: user.id,
    },
  });
}
```

Never rely only on:

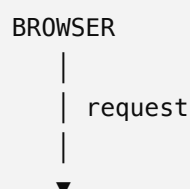
"the button isn't visible"

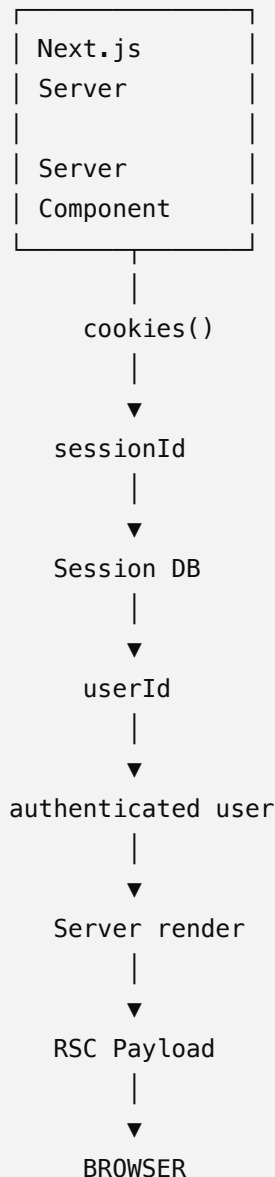


for security.

7.5 — The mental model

This is what I want you to visualize:





The **authentication decision** happens on the server.

Interview question

"How do you authenticate a user in a Next.js Server Component?"

A strong answer:

"Since Server Components execute on the server, I can read the request's session cookie using Next.js's server-side `cookies()` API. I use the session ID to look up and validate the session in the server-side session store, retrieve the associated user, and then render the appropriate content or redirect unauthenticated users. Sensitive authentication logic and database access remain on the server."

That's a very solid answer.

One subtle question before we move to 7.6

You may now be thinking:

"Okay, Server Components can access cookies directly because they run on the server. But what happens when a Client Component needs to know whether I'm logged in?"

That's exactly the problem **7.6 — Client Components + Authentication** solves.

And there are **several different approaches**, including passing safe user data from a Server Component, using a session endpoint, or using an auth library/provider.

That's where things get more interesting. 🔥



yes and also browser stores cookie right but since it passes through nextjs server so it can also access it?

Exactly. 🎯 You've got the right idea. The browser **stores the cookie**, but when it makes a request to your Next.js server, the browser **sends the applicable cookie along with that request**. Next.js can then read it.

Let's make the distinction crystal clear.



Where does the cookie actually live?

After login, the server sends:

```
</> http
```

```
Set-Cookie: sessionId=ABC123; HttpOnly; Secure; SameSite=Lax
```



The browser receives that response and stores:

Browser Cookie Storage

```
sessionId = ABC123
```



So the cookie is **stored by the browser**, not inside Next.js.

Then what happens on the next request?

Suppose the user visits:

/dashboard



The browser makes:

`</> http`



GET /dashboard

Cookie: sessionId=ABC123

Conceptually:



So **yes**:

The browser owns/stores the cookie, but the Next.js server can access the cookie value because the browser sends it with the request.

 **And HttpOnly doesn't prevent the server from reading it**

This is an important distinction.

If we have:

```
</> http
```

```
Set-Cookie: sessionId=ABC123; HttpOnly
```

then:

Browser JavaScript

```
</> JavaScript
```

```
document.cookie
```

✗ cannot access the `sessionId` cookie.

Next.js server

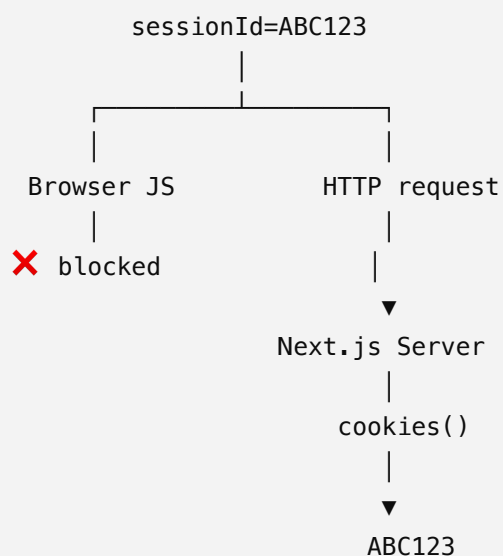
```
</> TypeScript
```

```
const cookieStore = await cookies();
```

```
const sessionId = cookieStore.get("sessionId")?.value;
```

✓ can access it.

So:



That's exactly why `HttpOnly` is useful.



One subtle correction

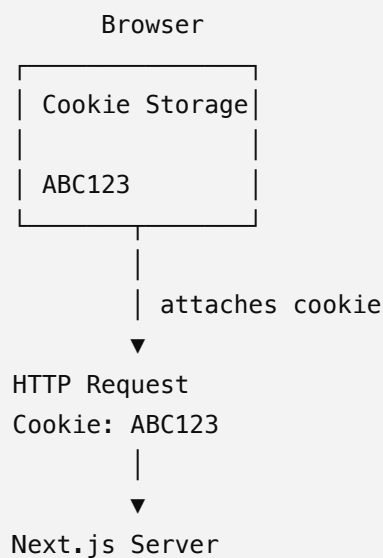
It's not quite:

```
Browser
  ↓
Next.js server
  ↓
Cookie
```



The cookie isn't *inside* the Next.js server.

It's more like:



The server gets a **copy** of the cookie value as part of the request.

What happens on the initial login request?

There's another subtle difference.

Before login

Browser:

```
No session cookie
```



User submits:

```
email = gopi@example.com
password = hello123
```



Request:

```
Browser
  |
  | email + password
  ▼
Next.js
```



Server authenticates and responds:

```
</> http

Set-Cookie: sessionId=ABC123; HttpOnly; Secure
```



Browser receives it and stores it.

Every later request

```
Browser
  |
  | Cookie: sessionId=ABC123
  ▼
Next.js
```



So the **first login establishes the cookie**, and subsequent requests use it.

And this explains our `getCurrentUser()` perfectly

When you do:

```
</> TypeScript

const user = await getCurrentUser();
```



inside a Server Component:

```
getCurrentUser()
  ↓
cookies()
  ↓
sessionId = ABC123
  ↓
DB/session store
  ↓
```



```
userId = 42
```



```
User
```

The server isn't somehow reaching into the browser's cookie storage.

Instead:

The browser already attached the cookie to the HTTP request, and Next.js reads that request cookie.

That's the exact mental model. 🔥

One sentence to lock it in

Cookie is stored by the browser → browser automatically sends it with matching requests → Next.js receives it in the request → `cookies()` lets server-side code read it.

And because it's `HttpOnly` :

Browser JavaScript can't read it, but the server can. 🍪🔒





Awesome 😊 Let's move to the **next important Auth.js concept**: how the session is actually used to protect pages and APIs.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

Google

↓

OAuth/OIDC

↓

Auth.js callback

↓

User/Account



↓
Session
↓
Cookie

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/  
├─ login/  
│   └─ page.tsx  
└─ dashboard/  
    └─ page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
</> TypeScript  
  
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}</h1>  
      <p>{session.user?.email}</p>  
    </div>  
  );  
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http  
  
GET /dashboard  
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript  
  
const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript  
  
// auth.ts  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:

```
Browser  
|
```




```
    | Cookie: session-token=ABC123
    ▼
  Next.js
    |
    ▼
Dashboard Server Component
    |
    | auth()
    ▼
Auth.js
    |
    ▼
Session verification / lookup
    |
    ▼
User session
```

If the session is valid:

</> TypeScript



```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

</> TypeScript



```
const session = await auth();
```

returns something equivalent to:

</> TypeScript



```
null
```

So:

```
</> TypeScript

if (!session) {
  redirect("/login");
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http

307 Temporary Redirect
Location: /login
```



The browser then requests:

```
</> http

GET /login
```



So:

```
User → /dashboard
      ↓
      auth()
      ↓
      no session
      ↓
      redirect("/login")
      ↓
      /login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript

if (!session)
```



That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

</> TypeScript



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule 🚨

Never rely only on hiding UI.

For example:

</> TypeScript



```
{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```

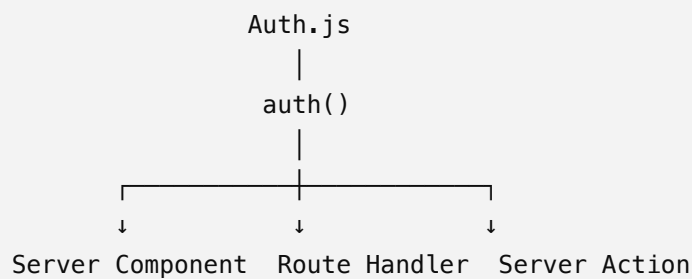


Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

`</>` TypeScript

```
const session = await auth();
```

Route Handler

`</>` TypeScript

```
const session = await auth();
```

Server Action

`</>` TypeScript

```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

```
</> TypeScript

"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and

authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:



```
1. LOGIN
  signIn()
    ↓
  OAuth
    ↓
  callback
    ↓
  session created

2. READING SESSION
  auth()
    ↓
  "Who is currently logged in?"

3. PROTECTING ACTIONS
  auth()
    ↓
  authenticate
    ↓
  authorize
    ↓
  perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

`</> TypeScript`



```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	Authorization / RBAC	Roles, permissions, admin/user access
7.10	Session Strategies	Database sessions vs JWT sessions
7.11	CSRF + SameSite	Why state-changing requests need protection

#	Topic	What you'll learn
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 🙌 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:



AUTHENTICATION

- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD
    Browser --> Login
    Login --> Authjs[Auth.js]
    Authjs --> Google[Google OAuth/OIDC]
    Google --> Callback
    Callback --> GoogleToken[Google token exchange]
    GoogleToken --> User[User + Account]
    User --> Adapter
    Adapter --> Database
    Database --> Session
    Session --> Cookie
    Cookie --> ProtectedRequest[Protected request]
    ProtectedRequest --> auth[auth()]
    auth --> Authentication
    Authentication --> Authorization
    Authorization --> ProtectedResource[Protected resource]
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅

📄 👍 ⬆️ ↺ ...

ChatGPT can make mistakes. Check important info.